

Esercizio 3 (10 punti).

Date le classi Tree, Branch e Leaf, riportate nel seguito, si consideri il codice della classe Esercizio3.

```
interface Tree {  
    public abstract boolean empty();  
    public abstract int max();  
    public Tree insert(int elem);  
    public Tree remove(int elem);  
} // end Tree
```

```
class Leaf implements Tree {  
  
    public boolean empty(){ return true; }  
  
    public int max() {  
        assert false;  
        return 0;  
    }  
  
    public Tree insert(int elem){  
        return new Branch(elem, this, this);  
    }  
  
    public Tree remove(int elem){  
        return this;  
    }  
} // end Leaf
```

```
class Branch implements Tree {  
    private int elem; private Tree left; private Tree right;  
  
    public Branch(int elem, Tree left, Tree right) {  
        this.elem = elem;  
        this.left = left; this.right = right;  
    }  
  
    public boolean empty(){ return false; }  
  
    public int max(){  
        return right.empty() ? elem : right.max();  
    }  
  
    public Tree insert(int elem) {  
        if (elem < this.elem)  
            left = left.insert(elem);  
        else if (elem > this.elem)  
            right = right.insert(elem);  
        return this;  
    }  
}
```

```

public Tree remove(int x) {
    if (x == elem) {
        if (left.empty())
            return right;
        if (right.empty())
            return left;
        elem = left.max();
        System.out.println( "CHECK POINT 1");
        left = left.remove(elem);
        return this;
    }
    else if (x < elem) {
        left = left.remove(x);
        return this;
    }
    else {
        right = right.remove(x);
        return this;
    }
}
} // end Branch

```

```

public class Esercizio3 {
    public static void main(String[] args) {
        Tree t = new Leaf();
        t = t.insert(3);
        t = t.insert(1);
        t = t.insert(4);
        t = t.insert(2);
        t = t.remove(3);
        System.out.println( "CHECK POINT 2");
    }
} // end Esercizio3

```

Si disegni una rappresentazione dello stato della memoria (Stack e Heap):

1. La prima volta che l'esecuzione raggiunge il check point 1;
2. Al check point 2.